UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería

Laboratorio 8

Gabriela Mazariegos
José Santiago Pereira
Brandon Javier Reyes

Task 2

Pregunta 2.1

Pregunta 2.1

El CTO de VisorShelf propone usar el detector original R-CNN (2014) para la primera versión del sistema. El equipo de ingeniería calcula que con el dataset actual y una CPU de tienda, cada imagen tardaría aproximadamente **45 segundos** en procesarse. Con esto responda en su reporte:

1. Identifique el cuello de botella principal de R-CNN que causa esa latencia. Explique por qué procesar 2,000 propuestas de región de forma independiente es computacionalmente costoso, conectando su respuesta con lo que la red hace internamente en cada pasada.
2. Fast R-CNN introdujo el **feature map compartido** y el **RoI Pooling** para resolver ese cuello de botella. Explique la lógica detrás de cada uno: ¿qué cómputo elimina el feature map compartido?, ¿qué problema resuelve el RoI Pooling y qué operación matemática realiza para producir un tensor de tamaño fijo a partir de regiones de tamaño variable?
3. Con Fast R-CNN el tiempo de CNN bajó a 0.3 s/imagen, pero el tiempo total seguía siendo ~2.3 s. Identifique el nuevo cuello de botella y explique por qué Selective Search representa un problema arquitectónico más profundo que simplemente ser lento.

1. **R**// En pocas palabras el cuello de botella en esta arquitectura es que esas 2000 propuestas se hacen independientemente cada una en la red neuronal, haciendo que el proceso sea muy costoso en términos computacionales. El problema es que muchas de estas regiones coinciden en gran parte entre sí, lo que significa que la red está recalculando convoluciones sobre los mismos píxeles una y otra vez sin utilizar ningún recurso.

2. **R**// En el caso de Fast R-CNN elimina la recursión sobre la CNN dado que ahora toma la imagen completa y la pasa enteramente en la CNN disminuyendo en el mismo ejemplo las 1999 pasadas restantes. En el caso de RoI Pooling, toma cada región de tamaño variable en el feature map, la divide en una matriz fija , y aplica max pooling en cada celda. Así siempre produce un tensor del mismo tamaño sin importar qué tan grande o pequeña sea la región original.

3. **R**// Selective search representa un problema ya que este es un algoritmo clásico que no corre en GPU, no es paralelizable, y no se puede entrenar end-to-end.

Pregunta 2.2

Pregunta 2.2

El equipo de VisorShelf decide usar Faster R-CNN como base del sistema. Un ingeniero junior pregunta: *"¿Por qué necesitamos una Region Proposal Network si ya tenemos el feature map? ¿No podríamos simplemente hacer sliding window directamente sobre el feature map?"*

Responda en su reporte:

4. Responda la pregunta del ingeniero junior. Explique qué hace la RPN que un sliding window clásico no puede hacer, y por qué el hecho de que la RPN opere sobre el mismo feature map del backbone es una ventaja semántica, no solo computacional.
5. La RPN utiliza **anchor boxes** predefinidos (9 por posición: 3 escalas × 3 relaciones de aspecto). Explique qué son los anchors y por qué la red predice **deltas** (Δx, Δy, Δw, Δh) en lugar de coordenadas absolutas. En la decodificación **w = w_a · e^(Δw)**, identifique qué representa cada símbolo y explique por qué se usa la exponencial para las dimensiones.
6. Faster R-CNN logra ~5 FPS con VGG16. La restricción de VisorShelf es procesar una imagen en menos de **500 ms** (≥ 2 FPS). Tomando en cuenta esa restricción, ¿recomendaría Faster R-CNN para producción en el hardware de tienda? Argumente su respuesta considerando al menos dos factores distintos a la velocidad pura (p. ej., precisión en objetos pequeños y densos, facilidad de fine-tuning, ecosistema de implementación).

4. **R**// Un sliding window es búsqueda bruta con ventana fija es decir no se adapta a escalas ni aprende. La RPN aprende a proponer regiones con distintas escalas y predice regresiones.

5. **R**// En w = w_a · e^(Δw): w es el ancho final, w_a el ancho del anchor, Δw el delta predicho. Se usa exponencial para garantizar que w sea siempre positivo y por ende que el ajuste sea simétrico.

6. **R**// Sí, lo recomendaría ya que Faster R-CNN es superior en objetos pequeños y densos gracias a su enfoque de dos etapas y tiene un ecosistema de tecnologías más completo para este caso.

Pregunta 2.3

Pregunta 2.3

El equipo de producto presenta dos propuestas para el detector de producción de VisorShelf

Dimensión	Propuesta A: Faster R-CNN + ResNet-50 + FPN	Propuesta B: YOLOv8n (nano) fine-tuned
Velocidad estimada	8–12 FPS en GPU / ~1.5 FPS en CPU	60–80 FPS en GPU / ~15 FPS en CPU
mAP@0.5 (referencia)	~55 en COCO	~37 en COCO
Tamaño del modelo	~160 MB	~6 MB
Manejo de objetos pequeños y densos	Alto (FPN multi-escala)	Moderado
Costo de fine-tuning	Moderado (pipeline de dos etapas)	Bajo (arquitectura simple)

Responda en su reporte:

7. Tomando en cuenta las restricciones operativas de VisorShelf (hardware sin GPU, latencia < 500 ms, anaqueles densos), ¿cuál propuesta recomendaría y por qué? No se limite a comparar los números de la tabla; argumente la lógica de la decisión conectándola con las características técnicas de cada arquitectura.
8. Si VisorShelf logra instalar una GPU de gama media (RTX 3060) en las tiendas flagship, ¿cambiaría su recomendación? Explique cómo ese cambio de hardware altera el trade-off entre las dos propuestas.
9. ¿Qué riesgo técnico específico introduce hacer fine-tuning de Faster R-CNN con un dataset de anaqueles sin aplicar learning rate diferenciado entre el backbone y las capas nuevas? ¿Cómo se llama ese fenómeno y cómo lo mitigaría?

7. **R**// Yo recomendaría YOLOv8n, dado que YOLO es un detector de una sola etapa que procesa todo en un solo forward pass, haciéndolo mucho más eficiente. Su modelo es viable para hardware de tienda limitado.

8. **R**// Sí, cambiaría a Faster R-CNN + FPN, dado que contamos con GPU, tengamos en cuenta que la GPU elimina el cuello de botella de CPU y permite aprovechar el FPN multi-escala, que es mucho mejor para detectar productos pequeños y densos en anaqueles.

9. **R**// El riesgo es el olvido catastrófico dado que si usamos el mismo learning rate las capas del backbone pre-entrenado se sobreescriben agresivamente con el dataset pequeño de anaqueles, afectando las representaciones aprendidas.